

Cours 15 — La recherche dichotomique

Semaine 15 ■ Manuel Hachette ch. 6 (§ 6.1, 6.2) ■ 2×1 h

Objectifs

- Comprendre pourquoi trier accélère la recherche
- Dérouler et programmer la recherche dichotomique
- Comparer les coûts : linéaire contre logarithmique

1. Pourquoi trier ?

Chercher un mot dans le dictionnaire papier : personne ne lit tout depuis la page 1. On ouvre au milieu, on compare, on élimine une moitié — parce que le dictionnaire est **trié**. C'était déjà la stratégie gagnante du « juste prix » (TP 2) : proposer 50, puis 25 ou 75...

Sur un tableau **non trié**, impossible de faire mieux que la recherche séquentielle du cours 11 : $O(n)$. Sur un tableau **trié**, on peut faire spectaculairement mieux.

2. La recherche dichotomique

Principe : maintenir une **zone de recherche** [gauche, droite] ; comparer x à l'élément du **milieu** ; éliminer la moitié inutile ; recommencer.

```
def dichotomie(t, x):
    """Renvoie un indice i tel que t[i] == x, ou None.

    Précondition : t est trié par ordre croissant.
    """
    gauche = 0
    droite = len(t) - 1
    while gauche <= droite:
        milieu = (gauche + droite) // 2
        if t[milieu] == x:
            return milieu
        elif t[milieu] < x:
            gauche = milieu + 1      # x est à droite
        else:
            droite = milieu - 1      # x est à gauche
    return None
```

Déroulé sur $t = [2, 5, 7, 11, 13, 17, 23]$ et $x = 17$:

gauche	droite	milieu	comparaison
0	6	3	$t[3] = 11 < 17$: à droite
4	6	5	$t[5] = 17$: trouvé , indice 5

Les trois pièges classiques : oublier $+ 1 / - 1$ (boucle infinie!), écrire $<$ au lieu de $<=$ dans la condition du **while** (rate les zones à un élément), appliquer la dichotomie à un tableau **non trié** (résultat faux sans erreur — d'où la précondition).

3. Combien d'étapes ? Le coût logarithmique

Chaque tour **divise par 2** la zone de recherche : n , puis $n/2$, $n/4$... jusqu'à 1. Le nombre de tours est le nombre de divisions par 2 possibles : environ $\log_2 n$ (l'exercice 4 de la semaine 2 le calculait déjà!).

taille n	séquentielle (pire cas)	dichotomie (pire cas)
100	100	7
1 000 000	1 000 000	20
1 milliard	1 milliard	30

30 comparaisons pour un milliard d'éléments : c'est la puissance du **coût logarithmique**, noté $O(\log n)$. Doubler la taille n'ajoute qu'**une** comparaison.

4. Le prix à payer

La dichotomie exige un tableau trié — et trier coûte cher (semaines 16-17). Le compromis : si on cherche **une seule fois**, la séquentielle suffit ; si on cherche **souvent** dans les mêmes données, trier d'abord est rentable. C'est exactement le choix du dictionnaire papier.

À retenir

- Dichotomie : zone [gauche, droite], milieu, éliminer une moitié — **sur tableau trié** seulement.
- `while gauche <= droite`; bornes mises à jour avec `milieu + 1` / `milieu - 1`.
- Coût $O(\log_2 n)$: $\times 2$ sur la taille = +1 comparaison ; 1 milliard $\rightarrow$ 30 étapes.
- Précondition « trié » écrite dans la docstring — sinon résultat faux silencieux.